

CSE 599k: LLM Serving Systems – Assignment 3

In this assignment, you will profile the performance of operations used in transformers and the transformer as a whole. The assignment consists of three parts: GEMM profiling, Attention Profiling and Transformer Profiling. Your submission will be in the form of a .zip file upload on Canvas, and must include a **short report with answers to the following questions (.pdf)** and the **code**.

Section 1: General Matrix Multiplication

Q1. 2 + 5 + 3 points

- What is the operational intensity of a GEMM in terms of M, N, K dimensions?
 - # of Multiplications: NMK
 - # of Additions: NMK
 - Total # of Operations: $2NMK$
 - # Bytes Moved: $2(MN+MK + NK)$
 - Assuming FP16
 - Operational intensity: $NMK/(MN+MK + NK)$
- Calculate the compute intensity for M in 128 - 2048 with step 128, $(N, K) = (512, 512)$, $(4096, 4096)$, $(14336, 4096)$, $(4096, 1024)$, $(1024, 4096)$ and show the result in a table. You can use <https://www.tablesgenerator.com> to generate tables.

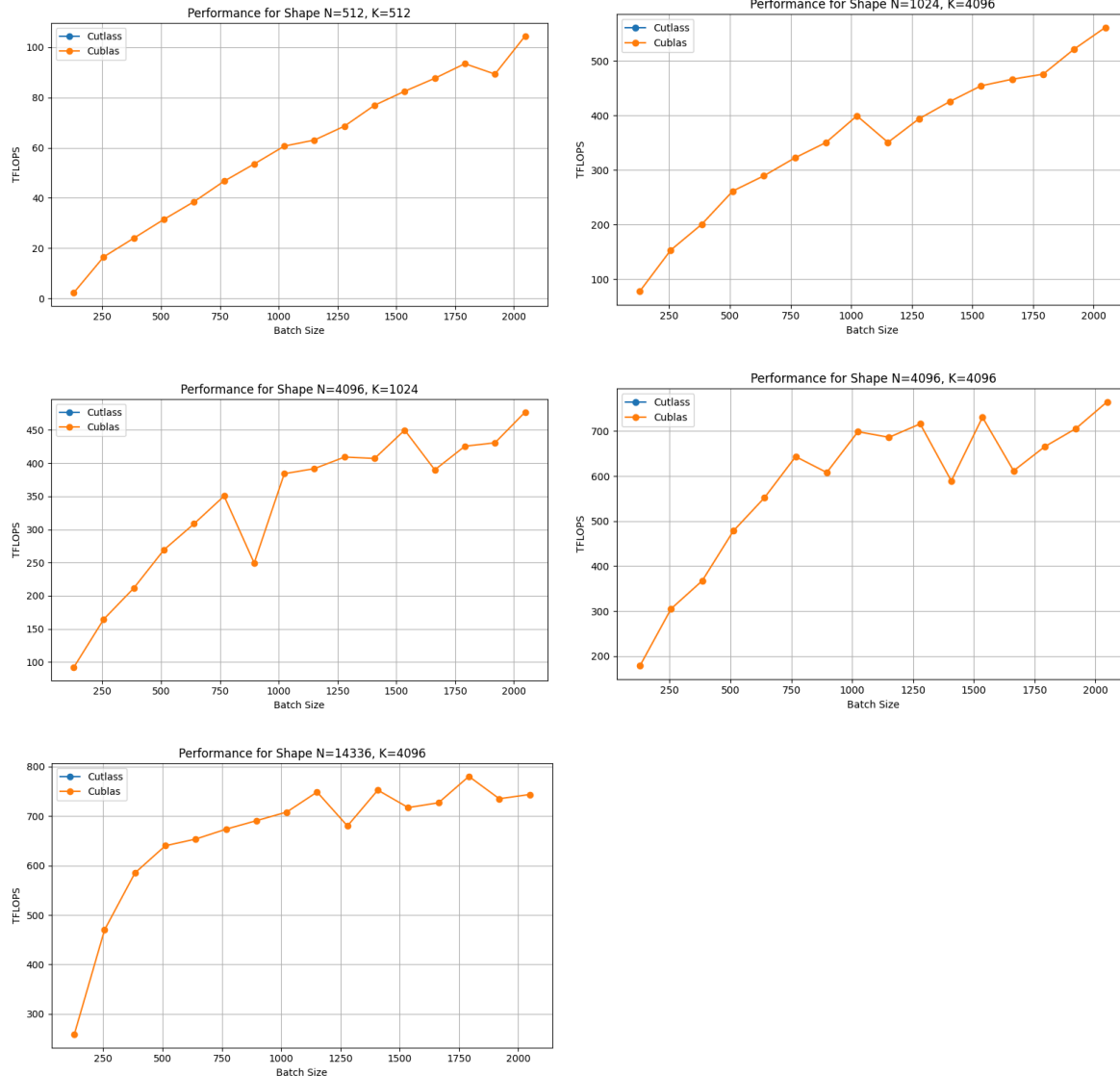
M	(512,512)	(4096,4096)	(14336,4096)	(4096,1024)	(1024,4096)
128	85.33	120.47	123.06	110.70	110.70
256	128.00	227.56	236.96	195.05	195.05
384	153.60	323.37	342.69	261.45	261.45
512	170.67	409.60	441.11	315.08	315.08
640	182.86	487.62	532.94	359.30	359.30
768	192.00	558.55	618.82	396.39	396.39
896	199.11	623.30	699.32	427.94	427.94
1024	204.80	682.67	774.92	455.11	455.11
1152	209.45	737.28	846.06	478.75	478.75
1280	213.33	787.69	913.12	499.51	499.51
1408	216.62	834.37	976.45	517.89	517.89
1536	219.43	877.71	1036.34	534.26	534.26
1664	221.87	918.07	1093.07	548.95	548.95
1792	224.00	955.73	1146.88	562.20	562.20
1920	225.88	990.97	1197.99	574.21	574.21
2048	227.56	1024.00	1246.61	585.14	585.14

-
- What is CUTLASS? What is CUBLAS? What is the key difference between them?
 - <https://docs.nvidia.com/cuda/cublas/>
 - <https://github.com/NVIDIA/cutlass>
 - CUTLASS is a open source collection of C++ templates used to implemented GEMM computations (and others, like convolutions) in CUDA. It is highly performant and allows high granularity / customization.
 - CUBLAS, on the other hand, is a closed source implementation of Nvidia's BLAS (linear algebra programs). CUBLAS's GEMM's are often more performant than CUTLASS's GEMM's.
 - A key difference between CUTLASS (open source) and CUBLAS (closed source) is that CUBLAS is often more performant, but CUTLASS, being open source, allows for experimentation and for users to modify (with great granularity) the operations and experiment with tiling.

Q2. 5 + 5 + 3 + 2 points

- Profile the CUBLAS cublasGEMMEx performance for the above shapes and plot the performance. Plot one figure for one shape with the M dimension on the x-axis and Tflops on the y-axis.
 - Please use the profiling methods in [Section1/prof_example.cu](#). We do warmups and L2 cache flush to increase the profiling accuracy.
 - The example plotting script is given at [Section1/plot_gemm.py](#).

Cublas Only:

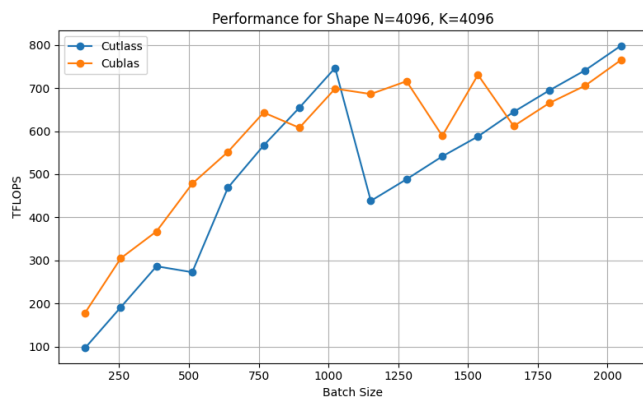
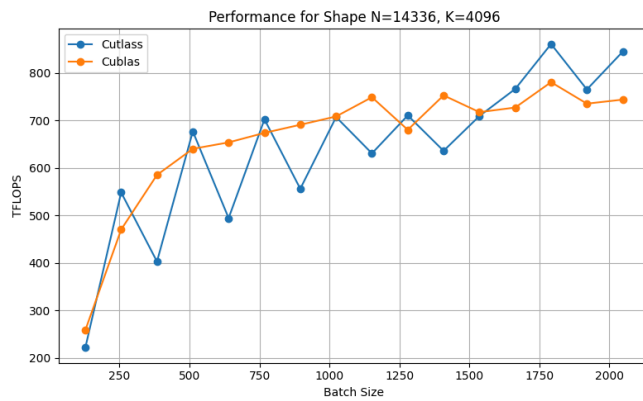


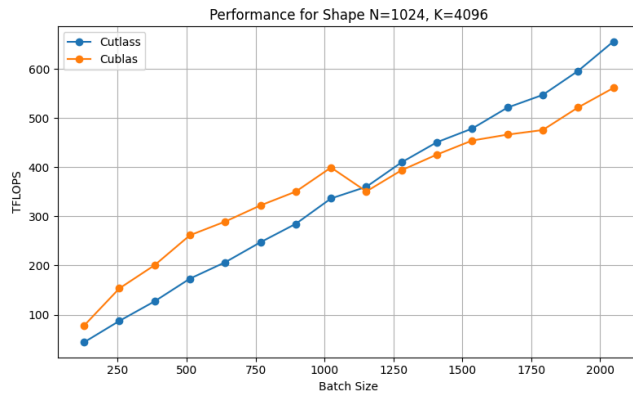
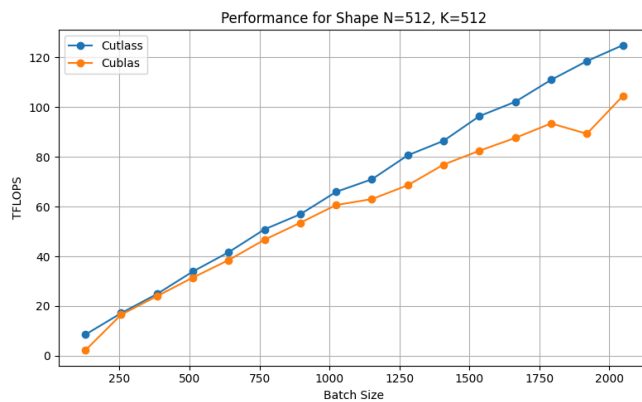
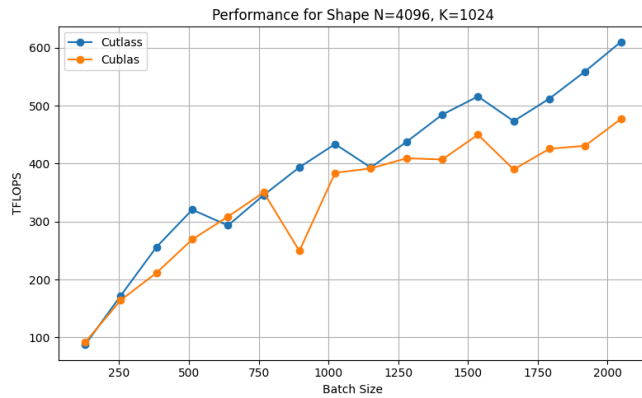
- Profile the CUTLASS GEMM performance using the CUTLASS profiler for the above shapes. Use again the `--profiling-iterations` of 100. Profile different `--split\_k\_slices`. Take

the best performance across all implementations. Plot the CUTLASS performance in the same figure as CUBLAS.

- <https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/quickstart.md>

Combined Cutlass and Cublas:





- Investigate the kernel that achieves the best performance in CUTLASS. What is the relationship between problem size, tile size, and split K?
 - https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/gemm_api_3x.md
 - The best kernels for the 5 shapes were
 - cutlass3x_sm90_tensorop_gemm_f16_f16_f32_f16_f16_128x256x64_2x1x1_0_nnn_align8_warpspecialized_cooperative_epi_tma
 - cutlass3x_sm90_tensorop_gemm_f16_f16_f32_f16_f16_128x256x64_2x1x1_0_nnn_align8_warpspecialized_pingpong_epi_tma

- What is the difference between the CUTLASS performance and the CUBLAS performance?
 - Cutlass generally outperforms Cublas. Cutlass is also generally more volatile than Cublas on larger values of N.
-

Section 2: Attention (currently incorrect, pending talk with TA)

Q1. 5 + 5 points

- Given model num_kv_heads, num_qo_heads, head_dim, what is the operational intensity of
 - Prefill attention, when the prompt length is p
 - # Operations:
 - Single Query Head: $4 * p^2 * \text{head_dim} + 3p^2$
 - Dot Product: $2 * p^2 * \text{head_dim}$
 - SoftMax: $p * (p + p + p) = 3p^2$
 - Condense: $2 * p^2 * \text{head_dim}$
 - Total # Ops: $\text{num_qo_heads} * (4 * p^2 * \text{head_dim} + 3p^2)$
 - # Bytes Moved:
 - Load KVQ: $(2 * \text{num_kv_heads} + \text{num_qo_heads}) * (\text{head_dim} * p)$
 - Output: $p * \text{head_dim} * \text{num_qo_heads}$
 - Total #: $2 * ((2 * \text{num_kv_heads} + \text{num_qo_heads}) * (\text{head_dim} * p) + p * \text{head_dim} * \text{num_qo_heads})$
 - Operational Intensity: # Operations / # Bytes Moved = $(\text{num_qo_heads} * (4 * p^2 * \text{head_dim} + 3p^2)) / (2 * ((2 * \text{num_kv_heads} + \text{num_qo_heads}) * (\text{head_dim} * p) + p * \text{head_dim} * \text{num_qo_heads}))$
 - Time Complexity: $O(p)$
 - Decode attention, when the context length is c
 - # Operations:
 - Single Query Head:
 - Dot Product: $2 * c * \text{head_dim}$
 - SoftMax: $3c$
 - Condense: $2 * c * \text{head_dim}$
 - Total # Ops: $(2 * c * \text{head_dim} + 3c + 2 * c * \text{head_dim}) * \text{num_qo_heads}$
 - # Bytes Moved:
 - Load KV: $2 * \text{num_kv_heads} * \text{head_dim} * c$
 - Load Q: $\text{num_qo_heads} * \text{head_dim}$
 - Output: $\text{head_dim} * \text{num_qo_heads}$
 - Total #: $2(2 * \text{num_kv_heads} * \text{head_dim} * c + \text{num_qo_heads} * \text{head_dim} + \text{head_dim} * \text{num_qo_heads})$

- Operational Intensity: $\# \text{ Operations} / \# \text{ Bytes Moved} = ((2 * c * \text{head_dim} + 3c + 2 * c * \text{head_dim}) * \text{num_qo_heads}) / (2(2 * \text{num_kv_heads} * \text{head_dim} * c + \text{num_qo_heads} * \text{head_dim} + \text{head_dim} * \text{num_qo_heads}))$
 - Time Complexity: $O(1)$
 - Compute the operational intensity for llama2-7B, llama3-8B, llama3-70B
 - llama2-7B:
 - Num_kv_heads: 32
 - Num_qo_heads: 32
 - Head_dim: 128
 - llama3-8B:
 - Num_kv_heads: 8
 - Num_qo_heads: 32
 - Head_dim: 128
 - llama3-70B:
 - Num_kv_heads: 8
 - Num_qo_heads: 64
 - Head_dim: 128
 - For prefill attention, $p = 2^7 - 2^{15}$

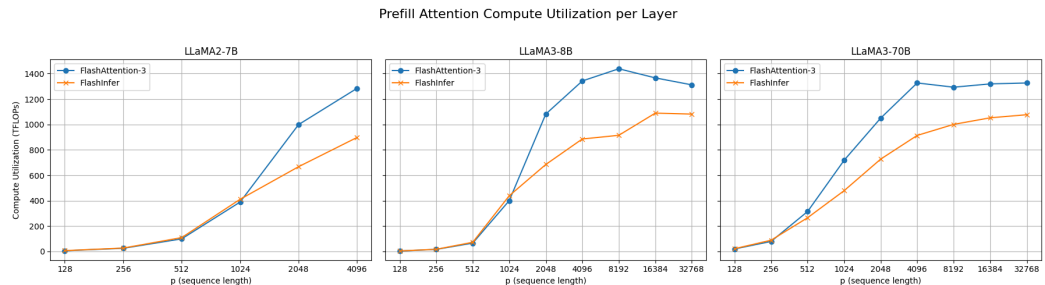
	A	B	C	D
1	p	llama2-7B	llama3-8B	llama3-70B
2	128	64.375	103.0	114.4444444444
3	256	128.75	206.0	228.8888888888
4	512	257.5	412.0	457.7777777777
5	1024	515.0	824.0	915.5555555555
6	2048	1030.0	1648.0	1831.1111111111
7	4096	2060.0	3296.0	3662.2222222222
8	8192	4120.0	6592.0	7324.4444444444
9	16384	8240.0	13184.0	14648.8888888888
10	32768	16480.0	26368.0	29297.7777777777

- For decode attention, $c = 2^7 - 2^{15}$

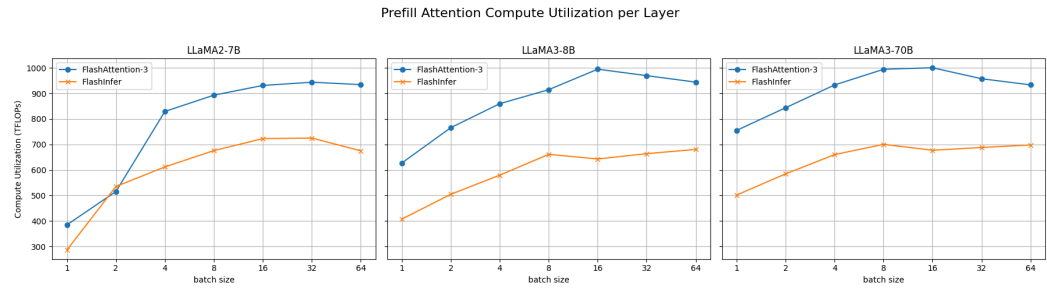
	A	B	C	D
1	c	llama2-7B	llama3-8B	llama3-70B
2	128	0.998062015503	3.901515151515	7.573529411764
3	256	1.001945525291	3.961538461538	7.803030303030
4	512	1.003898635477	3.992248062015	7.923076923076
5	1024	1.004878048780	4.007782101167	7.984496124031
6	2048	1.005368472425	4.015594541910	8.015564202334
7	4096	1.005613863802	4.019512195121	8.031189083820
8	8192	1.005736604418	4.021473889702	8.039024390243
9	16384	1.005797985962	4.022455455211	8.042947779404
10	32768	1.005828679544	4.022946417673	8.044910910422

Q2. 10 + 10 points

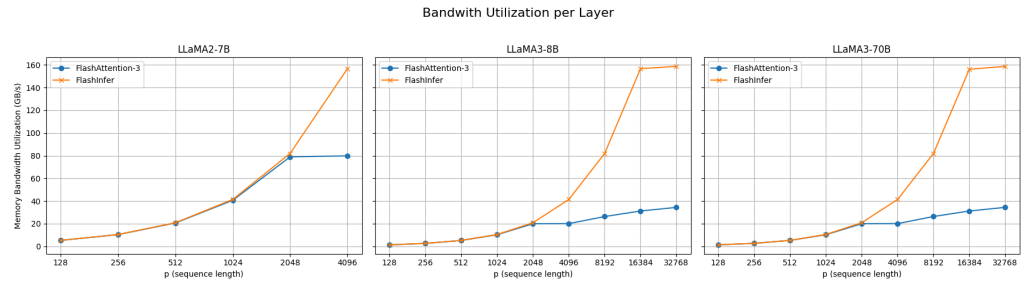
- Install FlashInfer using `uv pip install flashinfer-python==0.2.4 -i https://flashinfer.ai/whl/cu124/torch2.6/`.
- Install FlashAttention 3 by cloning the repo <https://github.com/Dao-AI-Lab/flash-attention> and run `cd hopper && python setup.py install`
- Evaluate the prefill attention performance of one layer using flash_attn3 <https://github.com/Dao-AI-Lab/flash-attention> and FlashInfer <https://github.com/flashinfer-ai/flashinfer/> for
 - Batch size = 1, $p = 2^7 - 2^{12}$ for llama2-7B, $p = 2^7 - 2^{15}$ for llama3-8B and llama3-70B. Plot the compute utilization TFlops of each model. Use $\log_2(p)$ as the x-axis. Generate one **subplot per model** inside a shared figure. In each subplot, draw line plots for both FlashAttention-3 and FlashInfer using different line styles or colors. Label axes clearly. (see [Section2/plot_attention_p.py](#))



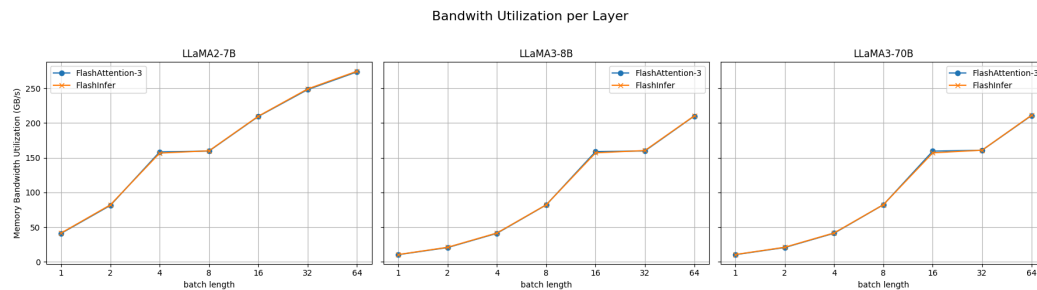
- Batch size = $2^0 - 2^6$, $p = 1024$ for all three models. Plot compute utilization using log batch size as the x-axis.



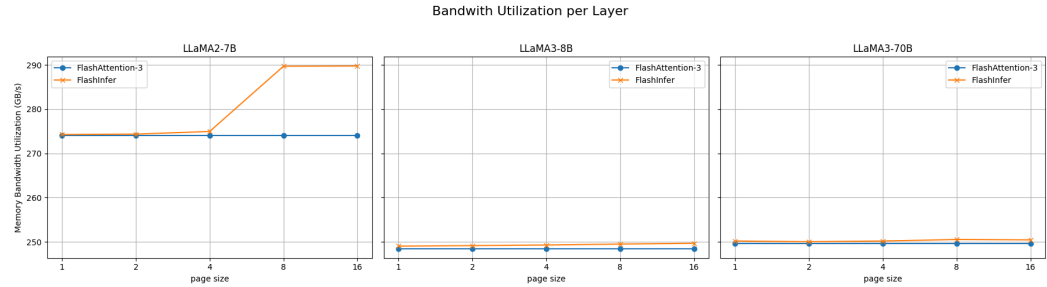
- Evaluate the decode attention performance of one layer with paged attention using flash_attn3 and Flashinfer
 - Batch size = 1, page size = 16, $c = 2^7 - 2^{12}$ for llama2-7B, $p = 2^7 - 2^{15}$ for llama3-8B and llama3-70B. Plot memory bandwidth utilization (GB/s) similar to previous questions.



- Batch size = $2^0 - 2^6$, $c = 1024$ for all three models. Plot memory bandwidth utilization similarly.



- Batch size = 128, $c = 1024$, page size = 1, 2, 4, 8, 16 for all three models. Plot the memory bandwidth utilization of each model.



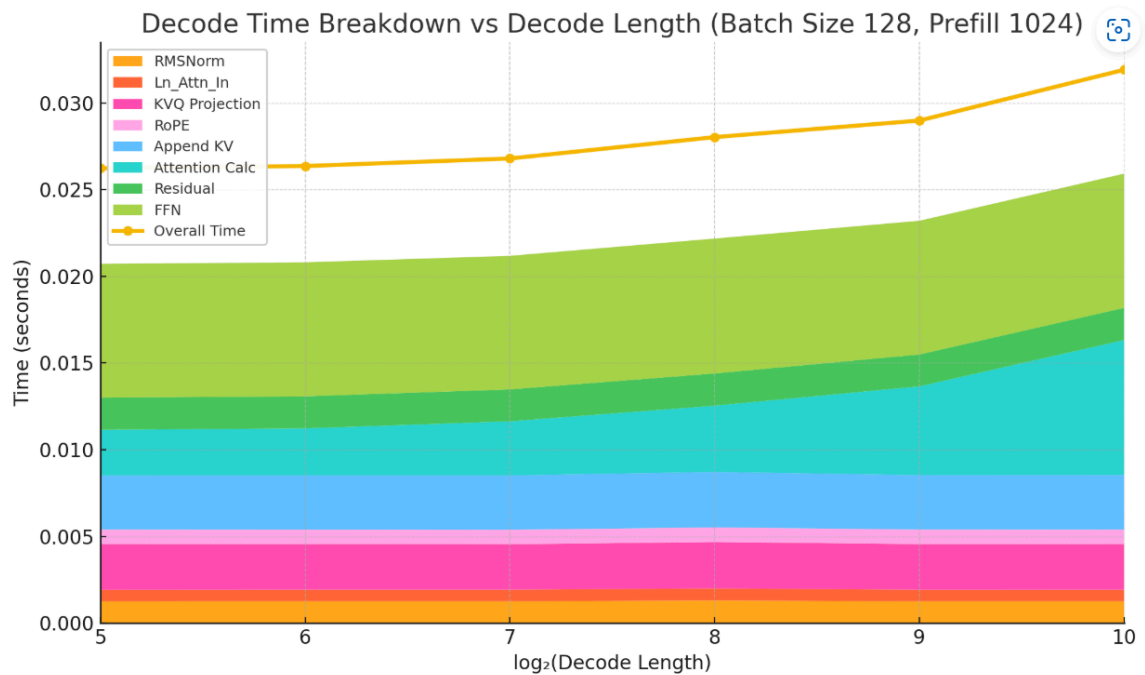
Section 3: End-to-end performance

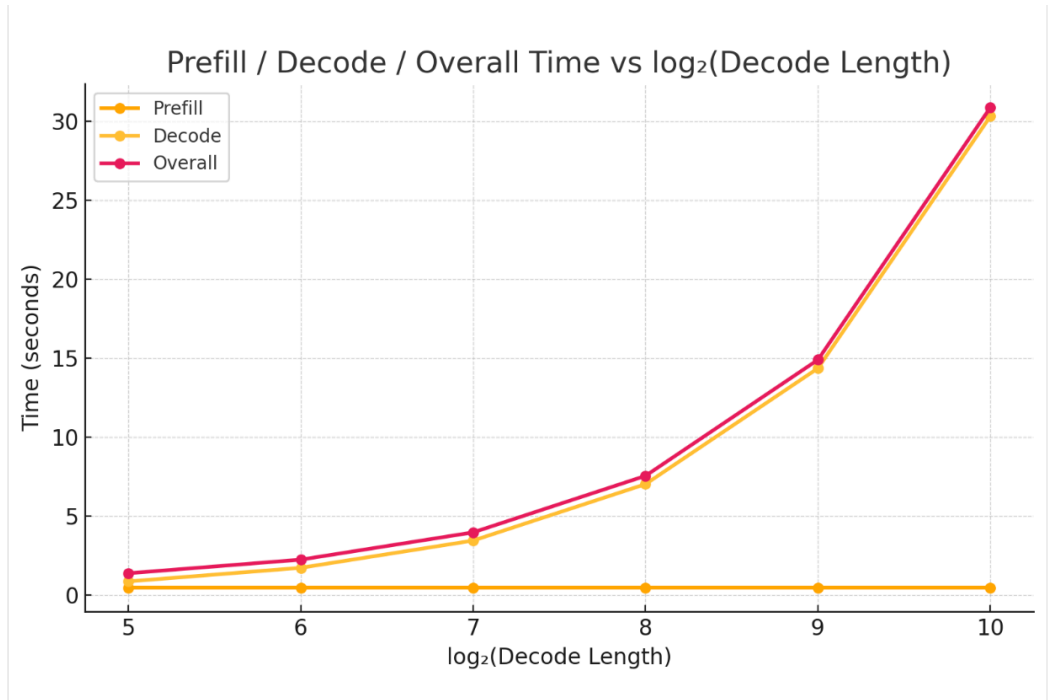
Q1. 15 points

- Starting from the [Section3/flashinfer_pipeline.py](#), construct a serving engine using FlashInfer attention and rope kernels with paged attention.

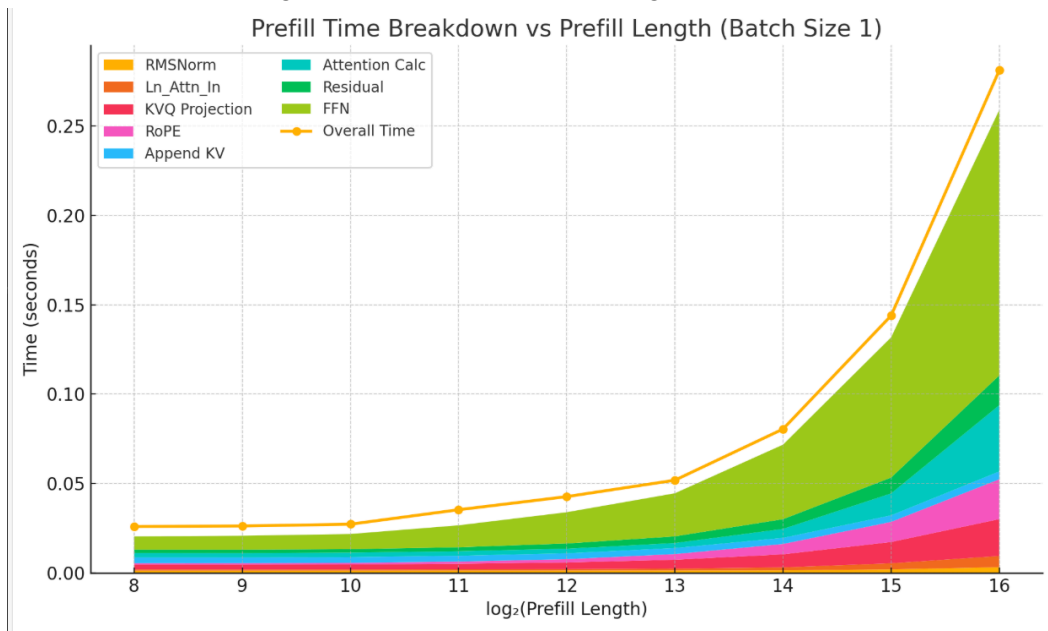
Q2. 10 + 10 + 10 points

- With batch size 128, prefill length 1024, decode length $2^5 - 2^{10}$, profile the prefill time and total decode time. Plot end-to-end time curve with the $\log_2(\text{decode length})$ as the x-axis. Which phase is the key bottleneck? In the last decode cycle, which operation takes the longest time for different decode lengths?





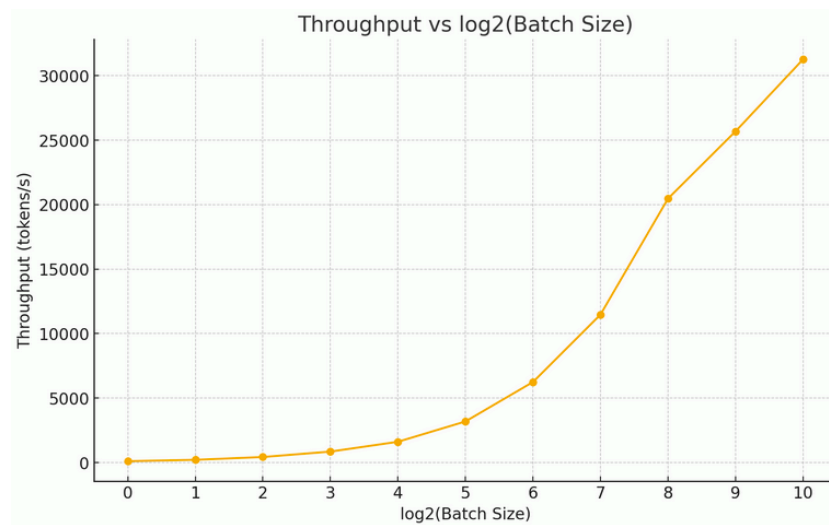
- The key bottleneck as decode length increases in the Decode operation
- In the decode cycle, the attention operation takes the longest time for different lengths as well as the FFN. Both of them are fairly constant, and don't increase quadratically with decode length.
- With batch size 1, prefill length $2^8 - 2^{16}$, profile the prefill time. Plot end-to-end time curve with log(prefill length) as x-axis. Examine the prefill time breakdown. What operations are the dominating factor for various prefill lengths?



- Looks like the dominating factors are FFN and Attention Calculation. It looks like they increase faster in proportion compared to the other operations as we

increase prefill lengths (with small prefill lengths, each operation is comparatively equal, but in larger prefill lengths, FFN and Attention Calculation dominate!)

- With batch size $2^0 - 2^{10}$, prefill length 128, decode length 128, plot the end-to-end time curve with $\log(\text{batch size})$ as x-axis. Plot the total throughput ((prefill + decode) / time) curve with $\log(\text{batch size})$ as x-axis. When does the performance saturate



- I think we would have to run the BatchSize for bigger ones, but it seems to start to saturate at 10 in the x-axis, and I think it would even out as we get to 12.